JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.stream

Interface Collector<T,A,R>

Type Parameters:

`T - the type of input elements to the reduction operation`

`A - the mutable accumulation type of the reduction operation (often hidden as an implementation detail)`

`R - the result type of the reduction operation`

`public interface` **`Collector<T,A,R>`**

A mutable reduction operation that accumulates input elements into a mutable result container, optionally transforming the accumulated result into a final representation after all input elements have been processed. Reduction operations can be performed either sequentially or in parallel.

Examples of mutable reduction operations include: accumulating elements into a `Collection`; concatenating strings using a `StringBuilder`; computing summary information about elements such as sum, min, max, or average; computing "pivot table" summaries such as "maximum valued transaction by seller", etc. The class `Collectors` provides implementations of many common mutable reductions.

A `Collector` is specified by four functions that work together to accumulate entries into a mutable result container, and optionally perform a final transform on the result. They are:

- creation of a new result container (`supplier()`)
- incorporating a new data element into a result container (`accumulator()`)
- combining two result containers into one (`combiner()`)
- performing an optional final transform on the container (`finisher()`)

Collectors also have a set of characteristics, such as `Collector.Characteristics.CONCURRENT`, that provide hints that can be used by a reduction implementation to provide better performance.

A sequential implementation of a reduction using a collector would create a single result container using the supplier function, and invoke the accumulator function once for each input element. A parallel implementation would partition the input, create a result container for each partition, accumulate the contents of each partition into a subresult for that partition, and then use the combiner function to merge the subresults into a combined result.

To ensure that sequential and parallel executions produce equivalent results, the collector functions must satisfy an *identity* and an associativity constraints.

The identity constraint says that for any partially accumulated result, combining it with an empty result container must produce an equivalent result. That is, for a partially accumulated result `a` that is the result of any series of accumulator and combiner invocations, `a` must be equivalent to `combiner.apply(a, supplier.get())`.

The associativity constraint says that splitting the computation must produce an equivalent result. That is, for any input elements `t1` and `t2`, the results `r1` and `r2` in the computation below must be equivalent:

```
A a1 = supplier.get();
accumulator.accept(a1, t1);
accumulator.accept(a1, t2);
R r1 = finisher.apply(a1);  // result without splitting

A a2 = supplier.get();
accumulator.accept(a2, t1);
A a3 = supplier.get();
accumulator.accept(a3, t2);
R r2 = finisher.apply(combiner.apply(a2, a3));  // result with splitting
```

For collectors that do not have the `UNORDERED` characteristic, two accumulated results `a1` and `a2` are equivalent if `finisher.apply(a1).equals(finisher.apply(a2))`. For unordered collectors, equivalence is relaxed to allow for non-equality related to differences in order. (For example, an unordered collector that accumulated elements to a `List` would consider two lists equivalent if they contained the same elements, ignoring order.)

Libraries that implement reduction based on `Collector`, such as `Stream.collect(Collector)`, must adhere to the following constraints:

- The first argument passed to the accumulator function, both arguments passed to the combiner function, and the argument passed to the finisher function must be the result of a previous invocation of the result supplier, accumulator, or combiner functions.

- The implementation should not do anything with the result of any of the result supplier, accumulator, or combiner functions other than to pass them again to the accumulator, combiner, or finisher functions, or return them to the caller of the reduction operation.
- If a result is passed to the combiner or finisher function, and the same object is not returned from that function, it is never used again.
- Once a result is passed to the combiner or finisher function, it is never passed to the accumulator function again.
- For non-concurrent collectors, any result returned from the result supplier, accumulator, or combiner functions must be serially thread-confined. This enables collection to occur in parallel without the `Collector` needing to implement any additional synchronization. The reduction implementation must manage that the input is properly partitioned, that partitions are processed in isolation, and combining happens only after accumulation is complete.
- For concurrent collectors, an implementation is free to (but not required to) implement reduction concurrently. A concurrent reduction is one where the accumulator function is called concurrently from multiple threads, using the same concurrently-modifiable result container, rather than keeping the result isolated during accumulation. A concurrent reduction should only be applied if the collector has the `Collector.Characteristics.UNORDERED` characteristics or if the originating data is unordered.

In addition to the predefined implementations in `Collectors`, the static factory methods `of(Supplier, BiConsumer, BinaryOperator, Characteristics...)` can be used to construct collectors. For example, you could create a collector that accumulates widgets into a `TreeSet` with:

```
Collector<Widget, ?, TreeSet<Widget>> intoSet =
    Collector.of(TreeSet::new, TreeSet::add,
                 (left, right) -> { left.addAll(right); return left; });
```

(This behavior is also implemented by the predefined collector `Collectors.toCollection(Supplier)`).

API Note:

Performing a reduction operation with a Collector should produce a result equivalent to:

```
R container = collector.supplier().get();
for (T t : data)
    collector.accumulator().accept(container, t);
return collector.finisher().apply(container);
```

However, the library is free to partition the input, perform the reduction on the partitions, and then use the combiner function to combine the partial results to achieve a parallel reduction. (Depending

on the specific reduction operation, this may perform better or worse, depending on the relative cost of the accumulator and combiner functions.)

Collectors are designed to be *composed*; many of the methods in Collectors are functions that take a collector and produce a new collector. For example, given the following collector that computes the sum of the salaries of a stream of employees:

```
Collector<Employee, ?, Integer> summingSalaries
    = Collectors.summingInt(Employee::getSalary))
```

If we wanted to create a collector to tabulate the sum of salaries by department, we could reuse the "sum of salaries" logic using Collectors.groupingBy(Function, Collector):

```
Collector<Employee, ?, Map<Department, Integer>> summingSalariesByDept
    = Collectors.groupingBy(Employee::getDepartment, summingSalaries);
```

Since:

1.8

See Also:

Stream.collect(Collector), Collectors

Nested Class Summary

Nested Classes

Modifier and Type	Interface	Description
static class	**Collector.Characteristics**	Characteristics indicating properties of a Collector, which can be used to optimize reduction implementations.

Method Summary

Default M

Modifier and Type	Method	D

BiConsumer<A,T>	accumulator()	A in co
Set<Collector.Characteristics>	characteristics()	R C in of
BinaryOperator<A>	combiner()	A pa th
Function<A,R>	finisher()	P tr in ty R.
static <T,A,R> Collector<T,A,R>	of(Supplier<A> supplier, BiConsumer<A,T> accumulator, BinaryOperator<A> combiner, Function<A,R> finisher, Collector.Characteristics... characteristics)	R de su co fu
static <T,R> Collector<T,R,R>	of(Supplier<R> supplier, BiConsumer<R,T> accumulator, BinaryOperator<R> combiner, Collector.Characteristics... characteristics)	R de su co
Supplier<A>	supplier()	A re co

Method Detail

supplier

Supplier<A> supplier()

A function that creates and returns a new mutable result container.

Returns:

a function which returns a new, mutable result container

accumulator

`BiConsumer<A,T> accumulator()`

A function that folds a value into a mutable result container.

Returns:

`a function which folds a value into a mutable result container`

combiner

`BinaryOperator<A> combiner()`

A function that accepts two partial results and merges them. The combiner function may fold state from one argument into the other and return that, or may return a new result container.

Returns:

`a function which combines two partial results into a combined result`

finisher

`Function<A,R> finisher()`

Perform the final transformation from the intermediate accumulation type `A` to the final result type `R`.

If the characteristic `IDENTITY_TRANSFORM` is set, this function may be presumed to be an identity transform with an unchecked cast from `A` to `R`.

Returns:

`a function which transforms the intermediate result to the final result`

characteristics

`Set<Collector.Characteristics> characteristics()`

Returns a `Set` of `Collector.Characteristics` indicating the characteristics of this Collector. This set should be immutable.

Returns:

`an immutable set of collector characteristics`

of

```
static <T,R> Collector<T,R,R> of(Supplier<R> supplier,
                                 BiConsumer<R,T> accumulator,
                                 BinaryOperator<R> combiner,
                                 Collector.Characteristics... characteristics)
```

Returns a new `Collector` described by the given `supplier`, `accumulator`, and `combiner` functions. The resulting `Collector` has the `Collector.Characteristics.IDENTITY_FINISH` characteristic.

Type Parameters:

`T` - The type of input elements for the new collector

`R` - The type of intermediate accumulation result, and final result, for the new collector

Parameters:

`supplier` - The supplier function for the new collector

`accumulator` - The accumulator function for the new collector

`combiner` - The combiner function for the new collector

`characteristics` - The collector characteristics for the new collector

Returns:

the new `Collector`

Throws:

`NullPointerException` - if any argument is null

of

```
static <T,A,R> Collector<T,A,R> of(Supplier<A> supplier,
                                   BiConsumer<A,T> accumulator,
                                   BinaryOperator<A> combiner,
                                   Function<A,R> finisher,
                                   Collector.Characteristics... characteristics)
```

Returns a new `Collector` described by the given `supplier`, `accumulator`, `combiner`, and `finisher` functions.

Type Parameters:

`T` - The type of input elements for the new collector

`A` - The intermediate accumulation type of the new collector

`R` - The final result type of the new collector

Parameters:

supplier - The supplier function for the new collector

accumulator - The accumulator function for the new collector

combiner - The combiner function for the new collector

finisher - The finisher function for the new collector

characteristics - The collector characteristics for the new collector

Returns:

the new Collector

Throws:

NullPointerException - if any argument is null

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.